10.9.1. Local Diagnosis: `netstat`

Let's first mention the `netstat` command (in the *net-tools* package); it displays an instant summary of a machine's network activity. When invoked with no argument, this command lists all open connections; this list can be very verbose since it includes many Unix-domain sockets (widely used by daemons) which do not involve the network at all (for example, dbus communication, X11 traffic, and communications between virtual filesystems and the desktop).

Common invocations therefore use options that alter `netstat`'s behavior. The most frequently used options include:

- -t, which filters the results to only include TCP connections;
- -u, which works similarly for UDP connections; these options are not mutually exclusive, and one of them is enough to stop displaying Unix-domain connections;
- -a, to also list listening sockets (waiting for incoming connections);
- -n, to display the results numerically: IP addresses (no DNS resolution), port numbers (no aliases as defined in `/etc/services`) and user ids (no login names);
- -p, to list the processes involved; this option is only useful when `netstat` is run as root, since normal users will only see their own processes;
- -c, to continuously refresh the list of connections.

Other options, documented in the `netstat(8)` manual page, provide an even finer control over the displayed results. In practice, the first five options are so often used together that systems and network administrators practically acquired `netstat -tupan` as a reflex. Typical results, on a lightly loaded machine, may look like the following:

```
# netstat -tupan
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 0.0.0.0:111             0.0.0.0:*               LISTEN      397/rpcbind
tcp        0      0 0.0.0.0:22              0.0.0.0:*               LISTEN      431/sshd
tcp        0      0 0.0.0.0:36568           0.0.0.0:*               LISTEN      407/rpc.statd
tcp        0      0 127.0.0.1:25            0.0.0.0:*               LISTEN      762/exim4
tcp        0    272 192.168.1.242:22        192.168.1.129:44452     ESTABLISHED 1172/sshd: roland [
tcp6       0      0 :::111                  :::*                    LISTEN      397/rpcbind
tcp6       0      0 :::22                   :::*                    LISTEN      431/sshd
tcp6       0      0 ::1:25                  :::*                    LISTEN      762/exim4
tcp6       0      0 :::35210                :::*                    LISTEN      407/rpc.statd
udp        0      0 0.0.0.0:39376           0.0.0.0:*                           916/dhclient
udp        0      0 0.0.0.0:996             0.0.0.0:*                           397/rpcbind
udp        0      0 127.0.0.1:1007          0.0.0.0:*                           407/rpc.statd
udp        0      0 0.0.0.0:68              0.0.0.0:*                           916/dhclient
udp        0      0 0.0.0.0:48720           0.0.0.0:*                           451/avahi-daemon: r
udp        0      0 0.0.0.0:111             0.0.0.0:*                           397/rpcbind
udp        0      0 192.168.1.242:123       0.0.0.0:*                           539/ntpd
udp        0      0 127.0.0.1:123           0.0.0.0:*                           539/ntpd
udp        0      0 0.0.0.0:123             0.0.0.0:*                           539/ntpd
udp        0      0 0.0.0.0:5353            0.0.0.0:*                           451/avahi-daemon: r
udp        0      0 0.0.0.0:39172           0.0.0.0:*                           407/rpc.statd
udp6       0      0 :::996                  :::*                                397/rpcbind
udp6       0      0 :::34277                :::*                                407/rpc.statd
udp6       0      0 :::54852                :::*                                916/dhclient
udp6       0      0 :::111                  :::*                                397/rpcbind
udp6       0      0 :::38007                :::*                                451/avahi-daemon: r
udp6       0      0 fe80::5054:ff:fe99::123 :::*                                539/ntpd
```

```
udp6       0      0 2001:bc8:3a7e:210:a:123 :::*                            539/ntpd
udp6       0      0 2001:bc8:3a7e:210:5:123 :::*                            539/ntpd
udp6       0      0 ::1:123                 :::*                            539/ntpd
udp6       0      0 :::123                  :::*                            539/ntpd
udp6       0      0 :::5353                 :::*                            451/avahi-daemon: r
```

As expected, this lists established connections, two SSH connections in this case, and applications waiting for incoming connections (listed as LISTEN), notably the Exim4 email server listening on port 25.

10.9.2. Remote Diagnosis: nmap

`nmap` (in the similarly-named package) is, in a way, the remote equivalent for `netstat`. It can scan a set of "well-known" ports for one or several remote servers, and list the ports where an application is found to answer to incoming connections. Furthermore, `nmap` is able to identify some of these applications, sometimes even their version number. The counterpart of this tool is that, since it runs remotely, it cannot provide information on processes or users; however, it can operate on several targets at once.

A typical `nmap` invocation only uses the -A option (so that `nmap` attempts to identify the versions of the server software it finds) followed by one or more IP addresses or DNS names of machines to scan. Again, many more options exist to finely control the behavior of `nmap`; please refer to the documentation in the `nmap(1)` manual page.

```
# nmap mirtuel

Starting Nmap 7.70 ( https://nmap.org ) at 2019-06-30 21:05 CET
Nmap scan report for mirtuel (192.168.1.242)
Host is up (0.000013s latency).
rDNS record for 192.168.1.242: mirtuel.internal.placard.fr.eu.org
Not shown: 998 closed ports
PORT    STATE SERVICE
22/tcp  open  ssh
111/tcp open  rpcbind

Nmap done: 1 IP address (1 host up) scanned in 2.41 seconds
# nmap -A localhost

Starting Nmap 7.70 ( https://nmap.org ) at 2019-06-30 21:17 CEST
Nmap scan report for localhost (127.0.0.1)
Host is up (0.000039s latency).
Other addresses for localhost (not scanned): ::1
Not shown: 997 closed ports
PORT    STATE SERVICE VERSION
22/tcp  open  ssh     OpenSSH 7.9p1 Debian 10 (protocol 2.0)
| ssh-hostkey:
|   2048 33:a1:d8:b1:e5:5b:b2:0d:15:1b:8e:76:7f:e4:d7:3d (RSA)
|   256 8f:83:cf:fa:b3:58:54:9a:1d:1b:4c:db:b1:e2:58:76 (ECDSA)
|_  256 fa:3d:58:62:49:92:93:90:52:fe:f4:26:ca:dc:4c:40 (ED25519)
```

```
25/tcp  open  smtp    Exim smtpd 4.92
| smtp-commands: mirtuel Hello localhost [127.0.0.1], SIZE 52428800, 8BITMIME,
    ➥ PIPELINING, CHUNKING, PRDR, HELP,
|_ Commands supported: AUTH HELO EHLO MAIL RCPT DATA BDAT NOOP QUIT RSET HELP
631/tcp open  ipp     CUPS 2.2
| http-methods:
|_  Potentially risky methods: PUT
| http-robots.txt: 1 disallowed entry
|_/
|_http-server-header: CUPS/2.2 IPP/2.1
|_http-title: Home - CUPS 2.2.10
Device type: general purpose
Running: Linux 3.X
OS CPE: cpe:/o:linux:linux_kernel:3
OS details: Linux 3.7 - 3.10
Network Distance: 0 hops
Service Info: Host: debian; OS: Linux; CPE: cpe:/o:linux:linux_kernel

OS and Service detection performed. Please report any incorrect results at https
    ➥ ://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 12.33 seconds
```

As expected, the SSH and Exim4 applications are listed. Note that not all applications listen on all IP addresses; since Exim4 is only accessible on the lo loopback interface, it only appears during an analysis of localhost and not when scanning mirtuel (which maps to the eth0 interface on the same machine).

10.9.3. Sniffers: `tcpdump` and `wireshark`

Sometimes, one needs to look at what actually goes on the wire, packet by packet. These cases call for a "frame analyzer", more widely known as a *sniffer*. Such a tool observes all the packets that reach a given network interface, and displays them in a user-friendly way.

The venerable tool in this domain is `tcpdump`, available as a standard tool on a wide range of platforms. It allows many kinds of network traffic capture, but the representation of this traffic stays rather obscure. We will therefore not describe it in further detail.

A more recent (and more modern) tool, `wireshark` (in the *wireshark* package), has become the new reference in network traffic analysis due to its many decoding modules that allow for a simplified analysis of the captured packets. The packets are displayed graphically with an organization based on the protocol layers. This allows a user to visualize all protocols involved in a packet. For example, given a packet containing an HTTP request, `wireshark` displays, separately, the information concerning the physical layer, the Ethernet layer, the IP packet information, the TCP connection parameters, and finally the HTTP request itself.

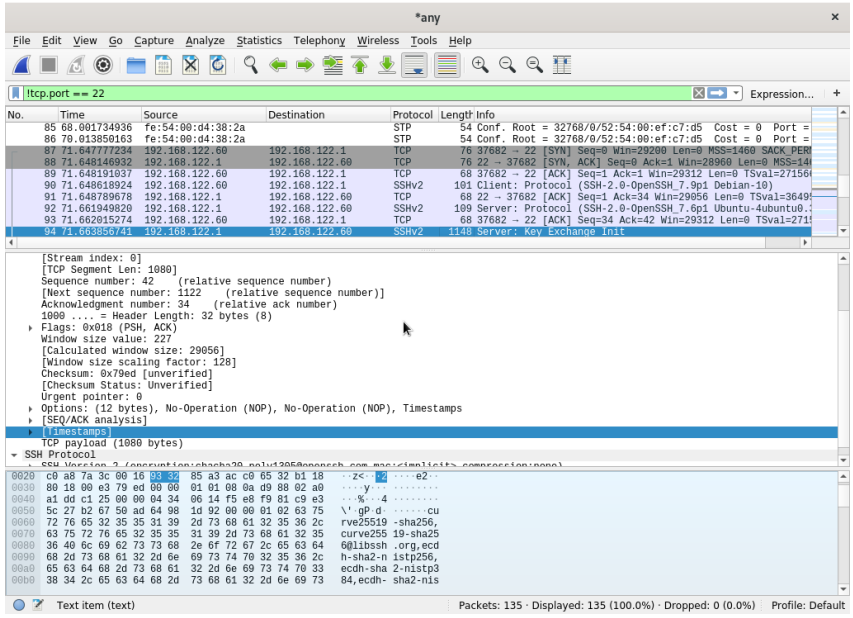


Figure 10.1 *The `wireshark` network traffic analyzer*

In our example, the packets traveling over SSH are filtered out (with the !tcp.port == 22 filter). The packet currently displayed was developed at the transport layer of the SSHv2 protocol.

TIP

`wireshark` with no graphical interface: `tshark`

When one cannot run a graphical interface, or does not wish to do so for whatever reason, a text-only version of `wireshark` also exists under the name `tshark` (in a separate *tshark* package). Most of the capture and decoding features are still available, but the lack of a graphical interface necessarily limits the interactions with the program (filtering packets after they've been captured, tracking of a given TCP connection, and so on). It can still be used as a first approach. If further manipulations are intended and require the graphical interface, the packets can be saved to a file and this file can be loaded into a graphical `wireshark` running on another machine.



Keywords

Postfix
Apache
NFS
Samba
Squid
OpenLDAP
SIP
SSL
OpenDKIM
SPF



Chapter

11

Network Services: Postfix, Apache, NFS, Samba, Squid, LDAP, SIP, XMPP, TURN

Contents

Mail Server 272 Web Server (HTTP) 293 FTP File Server 301 NFS File Server 302
Setting Up Windows Shares with Samba 305 HTTP/FTP Proxy 308 LDAP Directory 310
Real-Time Communication Services 319

Network services are the programs that users interact with directly in their daily work. They are the tip of the information system iceberg, and this chapter focuses on them; the hidden parts they rely on are the infrastructure we already described. They usually require the encryption technology described in section 10.2, "X.509 certificates" page 240.

11.1. Mail Server

The Falcot Corp administrators selected Postfix for the electronic mail server, due to its reliability and its ease of configuration. Indeed, its design enforces that each task is implemented in a process with the minimum set of required permissions, which is a great mitigation measure against security problems.

ALTERNATIVE
The Exim4 server

Debian uses Exim4 as the default email server (which is why the initial installation includes Exim4). The configuration is provided by a separate package, *exim4-config*, and automatically customized based on the answers to a set of Debconf questions very similar to the questions asked by the *postfix* package.

The configuration can be either in one single file (`/etc/exim4/exim4.conf.template`) or split across a number of configuration snippets stored under `/etc/exim4/conf.d/`. In both cases, the files are used by `update-exim4.conf` as templates to generate `/var/lib/exim4/config.autogenerated`. The latter is the file used by Exim4. Thanks to this mechanism, values obtained through Exim's debconf configuration — which are stored in `/etc/exim4/update-exim4.conf.conf` — can be injected in Exim's configuration file, even when the administrator or another package has altered the default Exim configuration.

The Exim4 configuration file syntax has its peculiarities and its learning curve; however, once these peculiarities are understood, Exim4 is a very complete and powerful email server, as evidenced by the tens of pages of documentation.

➨ https://www.exim.org/docs.html

11.1.1. Installing Postfix

The *postfix* package includes the main SMTP daemon. Other packages (such as *postfix-ldap* and *postfix-pgsql*) add extra functionality to Postfix, including access to mapping databases. You should only install them if you know that you need them.

BACK TO BASICS
SMTP

SMTP (*Simple Mail Transfer Protocol*, RFC 5321) is the protocol used by mail servers to exchange and route emails.

Several Debconf questions are asked during the installation of the package. The answers allow generating a first version of the `/etc/postfix/main.cf` configuration file.

The first question deals with the type of setup. Only two of the proposed answers are relevant in case of an Internet-connected server, "Internet site" and "Internet with smarthost". The former is appropriate for a server that receives incoming email and sends outgoing email directly to its recipients, and is therefore well-adapted to the Falcot Corp case. The latter is appropriate for a server receiving incoming email normally, but that sends outgoing email through an intermediate SMTP server — the "smarthost" — rather than directly to the recipient's server. This is mostly useful for individuals with a dynamic IP address, since many email servers reject messages coming straight from such an IP address. In this case, the smarthost will usually be

the ISP's SMTP server, which is always configured to accept email coming from the ISP's customers and forward it appropriately. This setup (with a smarthost) is also relevant for servers that are not permanently connected to the internet, since it avoids having to manage a queue of undeliverable messages that need to be retried later.

VOCABULARY
ISP
ISP is the acronym for "Internet Service Provider". It covers an entity, often a commercial company, that provides Internet connections and the associated basic services (email, news and so on).

The second question deals with the full name of the machine, used to generate email addresses from a local user name; the full name of the machine ends up as the part after the at-sign ("@"). In the case of Falcot, the answer should be mail.falcot.com. This is the only question asked by default, but the configuration it leads to is not complete enough for the needs of Falcot, which is why the administrators run `dpkg-reconfigure postfix` so as to be able to customize more parameters.

One of the extra questions asks for all the domain names related to this machine. The default list includes its full name as well as a few synonyms for localhost, but the main falcot.com domain needs to be added by hand. More generally, this question should usually be answered with all the domain names for which this machine should serve as an MX server; in other words, all the domain names for which the DNS says that this machine will accept email. This information ends up in the mydestination variable of the main Postfix configuration file — `/etc/postfix/main.cf`.

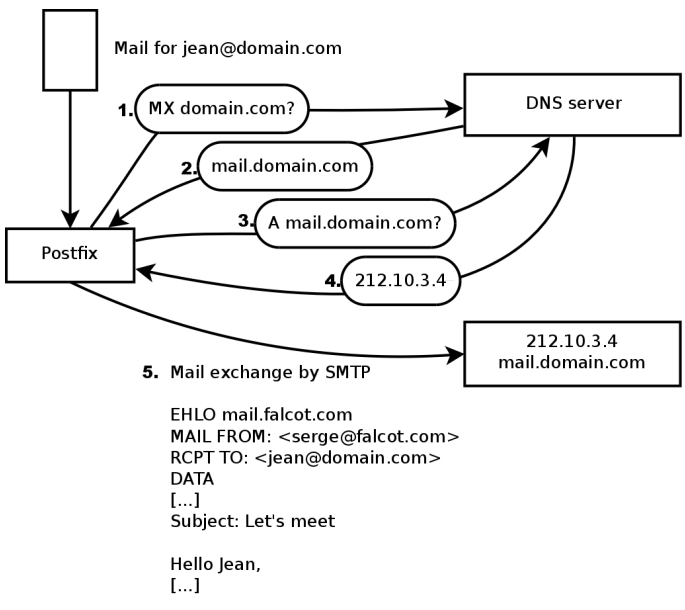


Figure 11.1 *Role of the DNS* MX *record while sending a mail*

EXTRA

Querying the MX records

When the DNS does not have an MX record for a domain, the email server will try sending the messages to the host itself, by using the matching A record (or AAAA in IPv6).

In some cases, the installation can also ask what networks should be allowed to send email via the machine. In its default configuration, Postfix only accepts emails coming from the machine itself; the local network will usually be added. The Falcot Corp administrators added 192.168.0.0/16 to the default answer. If the question is not asked, the relevant variable in the configuration file is mynetworks, as seen in the example below.

Local email can also be delivered through `procmail`. This tool allows users to sort their incoming email according to rules stored in their `~/.procmailrc` file. Both Postfix and Exim4 suggest *procmail* by default, but there are alternatives like *maildrop* or Sieve filters.

After this first step, the administrators got the following configuration file; it will be used as a starting point for adding some extra functionality in the next sections.

Example 11.1 *Initial `/etc/postfix/main.cf` file*

```
# See /usr/share/postfix/main.cf.dist for a commented, more complete version


# Debian specific:  Specifying a file name will cause the first
# line of that file to be used as the name.  The Debian default
# is /etc/mailname.
#myorigin = /etc/mailname

smtpd_banner = $myhostname ESMTP $mail_name (Debian/GNU)
biff = no

# appending .domain is the MUA's job.
append_dot_mydomain = no

# Uncomment the next line to generate "delayed mail" warnings
#delay_warning_time = 4h

readme_directory = no

# See http://www.postfix.org/COMPATIBILITY_README.html -- default to 2 on
# fresh installs.
compatibility_level = 2



# TLS parameters
smtpd_tls_cert_file=/etc/ssl/certs/ssl-cert-snakeoil.pem
smtpd_tls_key_file=/etc/ssl/private/ssl-cert-snakeoil.key
```

```
smtpd_use_tls=yes
smtpd_tls_session_cache_database = btree:${data_directory}/smtpd_scache
smtp_tls_session_cache_database = btree:${data_directory}/smtp_scache

# See /usr/share/doc/postfix/TLS_README.gz in the postfix-doc package for
# information on enabling SSL in the smtp client.

smtpd_relay_restrictions = permit_mynetworks permit_sasl_authenticated
    ➥ defer_unauth_destination
myhostname = mail.falcot.com
alias_maps = hash:/etc/aliases
alias_database = hash:/etc/aliases
myorigin = /etc/mailname
mydestination = mail.falcot.com, falcot.com, localhost.localdomain, localhost
relayhost =
mynetworks = 127.0.0.0/8 [::ffff:127.0.0.0]/104 [::1]/128 192.168.0.0/16
mailbox_size_limit = 0
recipient_delimiter = +
inet_interfaces = all
inet_protocols = all
```

SECURITY

Snake oil **SSL certificates**

The *snake oil* certificates, like the *snake oil* "medicine" sold by unscrupulous quacks in old times, have absolutely no value: you cannot rely on them to authenticate the server since they are automatically generated self-signed certificates. However, they are useful to improve the privacy of the exchanges.

In general they should only be used for testing purposes, and normal service must use real certificates. The Let's encrypt initiative offers free and trusted SSL/TLS certificates, which can be generated using the *certbot* package as described in section 11.2.2, "Adding support for SSL" page 294 and then used in `postfix` like this:

```
smtpd_tls_cert_file = /etc/letsencrypt/live/DOMAIN/
    ➥ fullchain.pem
smtpd_tls_key_file = /etc/letsencrypt/live/DOMAIN/privkey.
    ➥ pem
smtpd_tls_CAfile = /etc/ssl/certs/ca-certificates.crt
smtpd_tls_CApath = /etc/ssl/certs
smtp_tls_CApath = /etc/ssl/certs
```

A different way to generate own certificates is described in section 10.2.2, "Public Key Infrastructure: *easy-rsa*" page 243.

11.1.2. Configuring Virtual Domains

The mail server can receive emails addressed to other domains besides the main domain; these are then known as virtual domains. In most cases where this happens, the emails are not ul-

timately destined to local users. Postfix provides two interesting features for handling virtual domains.

CAUTION

Virtual domains and canonical domains

None of the virtual domains must be referenced in the `mydestination` variable; this variable only contains the names of the "canonical" domains directly associated to the machine and its local users.

Virtual Alias Domains

A virtual alias domain only contains aliases, i.e. addresses that only forward emails to other addresses.

Such a domain is enabled by adding its name to the virtual_alias_domains variable, and referencing an address mapping file in the virtual_alias_maps variable.

```
virtual_alias_domains = falcotsbrand.com
virtual_alias_maps = hash:/etc/postfix/virtual
```

The `/etc/postfix/virtual` file describes a mapping with a rather straightforward syntax: each line contains two fields separated by whitespace; the first field is the alias name, the second field is a list of email addresses where it redirects. The special @domain.com syntax covers all remaining aliases in a domain.

```
webmaster@falcotsbrand.com  jean@falcot.com
contact@falcotsbrand.com    laure@falcot.com, sophie@falcot.com
# The alias below is generic and covers all addresses within
# the falcotsbrand.com domain not otherwise covered by this file.
# These addresses forward email to the same user name in the
# falcot.com domain.
@falcotsbrand.com           @falcot.com
```

After changing `/etc/postfix/virtual` the postfix table `/etc/postfix/virtual.db` needs to be updated using `sudo postmap /etc/postfix/virtual`.

Virtual Mailbox Domains

CAUTION

Combined virtual domain?

Postfix does not allow using the same domain in both `virtual_alias_domains` and `virtual_mailbox_domains`. However, every domain of `virtual_mailbox_domains` is implicitly included in `virtual_alias_domains`, which makes it possible to mix aliases and mailboxes within a virtual domain.

Messages addressed to a virtual mailbox domain are stored in mailboxes not assigned to a local system user.

Enabling a virtual mailbox domain requires naming this domain in the virtual_mailbox_domains variable, and referencing a mailbox mapping file in virtual_mailbox_maps. The virtual_mailbox_base parameter contains the directory under which the mailboxes will be stored.

```
virtual_mailbox_domains = falcot.org
virtual_mailbox_maps = hash:/etc/postfix/vmailbox
virtual_mailbox_base = /var/mail/vhosts
```

The virtual_uid_maps parameter (respectively virtual_gid_maps) references the file containing the mapping between the email address and the system user (respectively group) that "owns" the corresponding mailbox. To get all mailboxes owned by the same owner/group, the static:5000 syntax assigns a fixed UID/GID (of value 5000 here).

Again, the syntax of the `/etc/postfix/vmailbox` file is quite straightforward: two fields separated with whitespace. The first field is an email address within one of the virtual domains, and the second field is the location of the associated mailbox (relative to the directory specified in *virtual_mailbox_base*). If the mailbox name ends with a slash (/), the emails will be stored in the *maildir* format; otherwise, the traditional *mbox* format will be used. The *maildir* format uses a whole directory to store a mailbox, each individual message being stored in a separate file. In the *mbox* format, on the other hand, the whole mailbox is stored in one file, and each line starting with "From " (From followed by a space) signals the start of a new message.

```
# Jean's email is stored as maildir, with
# one file per email in a dedicated directory
jean@falcot.org falcot.org/jean/
# Sophie's email is stored in a traditional "mbox" file,
# with all mails concatenated into one single file
sophie@falcot.org falcot.org/sophie
```

11.1.3. Restrictions for Receiving and Sending

The growing number of unsolicited bulk emails (*spam*) requires being increasingly strict when deciding which emails a server should accept. This section presents some of the strategies included in Postfix.

If the reject-rules are too strict, it may happen that even legitimate email traffic gets locked out. It is therefor a good habit to test restrictions and prevent the permanent rejection of requests during this time using the soft_bounce = yes directive. By prepending a reject-type directive with warn_if_reject only a log message will be recorded instead of rejecting the request.

CULTURE

The spam problem

"Spam" is a generic term used to designate all the unsolicited commercial emails (also known as UCEs) that flood our electronic mailboxes; the unscrupulous individuals sending them are known as spammers. They care little about the nuisance they cause, since sending an email costs very little, and only a very small percentage of recipients need to be attracted by the offers for the spamming operation to make more money than it costs. The process is mostly automated, and any email address made public (for instance, on a web forum, or on the archives of a mailing

list, or on a blog, and so on) will be likely discovered by the spammers' robots, and subjected to a never-ending stream of unsolicited messages. Also every contact found at a compromised system is targeted.

All system administrators try to face this nuisance with spam filters, but of course spammers keep adjusting to try to work around these filters. Some even rent networks of machines compromised by a worm from various crime syndicates. Recent statistics estimate that up to 95% of all emails circulating on the Internet are spam!

IP-Based Access Restrictions

The smtpd_client_restrictions directive controls which machines are allowed to communicate with the email server.

When a variable contains a list of rules, as in the example below, these rules are evaluated in order, from the first to the last. Each rule can accept the message, reject it, or leave the decision to a following rule. As a consequence, order matters, and simply switching two rules can lead to a widely different behavior.

Example 11.2 *Restrictions Based on Client Address*

```
smtpd_client_restrictions =
    permit_mynetworks,
    warn_if_reject reject_unknown_client_hostname,
    check_client_access hash:/etc/postfix/access_clientip,
    reject_rhsbl_reverse_client dbl.spamhaus.org,
    reject_rhsbl_reverse_client rhsbl.sorbs.net,
    reject_rbl_client zen.spamhaus.org,
    reject_rbl_client dnsbl.sorbs.net
```

The permit_mynetworks directive, used as the first rule, accepts all emails coming from a machine in the local network (as defined by the mynetworks configuration variable).

The second directive would normally reject emails coming from machines without a completely valid DNS configuration. Such a valid configuration means that the IP address can be resolved to a name, and that this name, in turn, resolves to the IP address. This restriction is often too strict, since many email servers do not have a reverse DNS for their IP address. This explains why the Falcot administrators prepended the warn_if_reject modifier to the reject_unknown_client directive: this modifier turns the rejection into a simple warning recorded in the logs. The administrators can then keep an eye on the number of messages that would be rejected if the rule were actually enforced, and make an informed decision later if they wish to enable such enforcement.

TIP

***access* tables**

The restriction criteria include administrator-modifiable tables listing combinations of senders, IP addresses, and allowed or forbidden hostnames. These tables can be created using an uncompressed copy of the /usr/share/doc/postfix/

examples/access.gz file shipped with the *postfix-doc* package. This model is self-documented in its comments, which means each table describes its own syntax.

The /etc/postfix/access_clientip table lists IP addresses and networks; /etc/postfix/access_helo lists domain names; /etc/postfix/access_sender contains sender email addresses. All these files need to be turned into hash-tables (a format optimized for fast access) after each change, with the sudo postmap /etc/postfix/*file* command.

The third directive allows the administrator to set up a blacklist and a whitelist of email servers, stored in the /etc/postfix/access_clientip file. Servers in the whitelist are considered as trusted, and the emails coming from there therefore do not go through the following filtering rules.

The last four rules reject any message coming from a server listed in one of the indicated blacklists. RBL is an acronym for *Remote Black List*, and RHSBL stands for *Right-Hand Side Black List*. The difference is, that the former lists IP addresses, whereas the latter lists domain names. There are several such services. They list domains and IP addresses with poor reputation, badly configured servers that spammers use to relay their emails, as well as unexpected mail relays such as machines infected with worms or viruses.

TIP

White list and RBLs

Blacklists sometimes include a legitimate server that has been suffering an incident. In these situations, all emails coming from one of these servers would be rejected unless the server is listed in a whitelist defined by /etc/postfix/access_clientip.

Prudence therefore recommends including in the whitelist(s) all the trusted servers from which many emails are usually received.

Checking the Validity of the EHLO or HELO Commands

Each SMTP exchange starts with a HELO (or EHLO) command, followed by the name of the sending email server. Checking the validity of this name can be interesting. To fully enforce the restrictions listed in smtpd_helo_restrictions the smtpd_helo_required option needs to be enabled. Otherwise clients could skip the restrictions by not sending any HELO/EHLO command.

Example 11.3 *Restrictions on the name announced in EHLO*

```
smtpd_helo_required = yes
smtpd_helo_restrictions =
    permit_mynetworks,
    reject_invalid_helo_hostname,
    reject_non_fqdn_helo_hostname,
    warn_if_reject reject_unknown_helo_hostname,
```

```
    check_helo_access hash:/etc/postfix/access_helo,
    reject_rhsbl_helo multi.surbl.org
```

The first permit_mynetworks directive allows all machines on the local network to introduce themselves freely. This is important, because some email programs do not respect this part of the SMTP protocol adequately enough, and they can introduce themselves with nonsensical names.

The reject_invalid_helo_hostname rule rejects emails when the EHLO announce lists a syntactically incorrect hostname. The reject_non_fqdn_helo_hostname rule rejects messages when the announced hostname is not a fully-qualified domain name (including a domain name as well as a host name). The reject_unknown_helo_hostname rule rejects messages if the announced name does not exist in the DNS. Since this last rule unfortunately leads to too many rejections, the administrators turned its effect to a simple warning with the warn_if_reject modifier as a first step; they may decide to remove this modifier at a later stage, after auditing the results of this rule.

The reject_rhsbl_helo allows to specify a black list to check the hostname against an RHSBL.

Using permit_mynetworks as the first rule has an interesting side effect: the following rules only apply to hosts outside the local network. This allows blacklisting all hosts that announce themselves as part of the falcot.com network, for instance by adding a falcot.com REJECT You are not in our network! line to the `/etc/postfix/access_helo` file.

Accepting or Refusing Based on the Announced Sender

Every message has a sender, announced by the MAIL FROM command of the SMTP protocol; again, this information can be validated in several different ways.

Example 11.4 *Sender checks*

```
smtpd_sender_restrictions =
    check_sender_access hash:/etc/postfix/access_sender,
    reject_unknown_sender_domain,
    reject_unlisted_sender,
    reject_non_fqdn_sender,
    reject_rhsbl_sender rhsbl.sorbs.net
```

The `/etc/postfix/access_sender` table maps some special treatment to some senders. This usually means listing some senders into a white list or a black list.

The reject_unknown_sender_domain rule requires a valid sender domain, since it is needed for a valid address. The reject_unlisted_sender rule rejects local senders if the address does not exist; this prevents emails from being sent from an invalid address in the falcot.com domain, and messages emanating from joe.bloggs@falcot.com are only accepted if such an address really exists.

Finally, the reject_non_fqdn_sender rule rejects emails purporting to come from addresses without a fully-qualified domain name. In practice, this means rejecting emails coming from user@machine: the address must be announced as either user@machine.example.com or user@example.com.

The reject_rhsbl_sender rule reject senders based on a (domain-based) RHSBL service.

Accepting or Refusing Based on the Recipient

Each email has at least one recipient, announced with the RCPT TO command in the SMTP protocol. These addresses also warrant validation, even if that may be less relevant than the checks made on the sender address.

Example 11.5 *Recipient checks*

```
smtpd_recipient_restrictions =
    permit_mynetworks,
    reject_unauth_destination,
    reject_unlisted_recipient,
    reject_non_fqdn_recipient,
    permit
```

reject_unauth_destination is the basic rule that requires outside messages to be addressed to us; messages sent to an address not served by this server are rejected. Without this rule, a server becomes an open relay that allows spammers to send unsolicited emails; this rule is therefore mandatory, and it will be best included near the beginning of the list, so that no other rules may authorize the message before its destination has been checked.

The reject_unlisted_recipient rule rejects messages sent to non-existing local users, which makes sense. Finally, the reject_non_fqdn_recipient rule rejects non-fully-qualified addresses; this makes it impossible to send an email to jean or jean@machine, and requires using the full address instead, such as jean@machine.falcot.com or jean@falcot.com.

The permit directive at the end is not necessary. But it can be useful at the end of a restriction list to make the default policy explicit.

Restrictions Associated with the DATA Command

The DATA command of SMTP is emitted before the contents of the message. It doesn't provide any information per se, apart from announcing what comes next. It can still be subjected to checks.

Example 11.6 *DATA checks*

```
smtpd_data_restrictions = reject_unauth_pipelining
```

The reject_unauth_pipelining directives causes the message to be rejected if the sending party sends a command before the reply to the previous command has been sent. This guards against a common optimization used by spammer robots, since they usually don't care a fig about replies and only focus on sending as many emails as possible in as short a time as possible.

Applying Restrictions

Although the above commands validate information at various stages of the SMTP exchange, Postfix sends the actual rejection as a reply to the RCPT TO command by default.

This means that even if the message is rejected due to an invalid EHLO command, Postfix knows the sender and the recipient when announcing the rejection. It can then log a more explicit message than it could if the transaction had been interrupted from the start. In addition, a number of SMTP clients do not expect failures on the early SMTP commands, and these clients will be less disturbed by this late rejection.

A final advantage to this choice is that the rules can accumulate information during the various stages of the SMTP exchange; this allows defining more fine-grained permissions, such as rejecting a non-local connection if it announces itself with a local sender.

The default behavior is controlled by the smtpd_delay_reject rule.

Filtering Based on the Message Contents

The validation and restriction system would not be complete without a way to apply checks to the message contents. Postfix differentiates the checks applying to the email headers from those applying to the email body.

Example 11.7 *Enabling content-based filters*

```
header_checks = regexp:/etc/postfix/header_checks
body_checks = regexp:/etc/postfix/body_checks
```

Both files contain a list of regular expressions (commonly known as *regexps* or *regexes*) and associated actions to be triggered when the email headers (or body) match the expression.

QUICK LOOK

Regexp tables

The file /usr/share/doc/postfix/examples/header_checks.gz (from the *postfix-doc* package) and header_checks(5) contain many explanatory comments and can be used as a starting point for creating the /etc/postfix/header_checks and /etc/postfix/body_checks files.

Example 11.8 *Example /etc/postfix/header_checks file*

```
/^X-Mailer: GOTO Sarbacane/ REJECT I fight spam (GOTO Sarbacane)
/^Subject: *Your email contains VIRUSES/ DISCARD virus notification
```

BACK TO BASICS

Regular expression

The *regular expression* term (shortened to *regexp* or *regex*) references a generic notation for expressing a description of the contents and/or structure of a string of characters. Certain special characters allow defining alternatives (for instance, `foo|bar` matches either "foo" or "bar"), sets of allowed characters (for instance, `[0-9]` means "any digit", and `.` — a dot — means "any character"), quantification (`s?` matches either `s` or the empty string, in other words 0 or 1 occurrence of `s`; `s+` matches one or more consecutive `s` characters; and so on). Parentheses allow grouping search results.

The precise syntax of these expressions varies across the tools using them, but the basic features are similar.

➨ https://en.wikipedia.org/wiki/Regular_expression

The first one checks the header mentioning the email software; if GOTO Sarbacane (a bulk email software) is found, the message is rejected. The second expression controls the message subject; if it mentions a virus notification, we can decide not to reject the message but to discard it immediately instead.

Using these filters is a double-edged sword, because it is easy to make the rules too generic and to lose legitimate emails as a consequence. In these cases, not only the messages will be lost, but their senders will get unwanted (and annoying) error messages.

11.1.4. Setting Up *greylisting*

"Greylisting" is a filtering technique according to which a message is initially rejected with a temporary error code, and only accepted on a further try after some delay. This filtering is particularly efficient against spam sent by the many machines infected by worms and viruses, since this software rarely acts as a full SMTP agent (by checking the error code and retrying failed messages later), especially since many of the harvested addresses are really invalid and retrying would only mean losing time.

Postfix doesn't provide greylisting natively, but there is a feature by which the decision to accept or reject a given message can be delegated to an external program. The *postgrey* package contains just such a program, designed to interface with this access policy delegation service.

Once *postgrey* is installed, it runs as a daemon and listens on port 10023. Postfix can then be configured to use it, by adding the check_policy_service parameter as an extra restriction:

```
smtpd_recipient_restrictions =
    permit_mynetworks,
    [...]
    check_policy_service inet:127.0.0.1:10023
```

Each time Postfix reaches this rule in the ruleset, it will connect to the `postgrey` daemon and send it information concerning the relevant message. On its side, Postgrey considers the IP address/sender/recipient triplet and checks in its database whether that same triplet has been seen recently. If so, Postgrey replies that the message should be accepted; if not, the reply indicates that the message should be temporarily rejected, and the triplet gets recorded in the database.

The main disadvantage of greylisting is that legitimate messages get delayed, which is not always acceptable. It also increases the burden on servers that send many legitimate emails.

IN PRACTICE

Shortcomings of greylisting

Theoretically, greylisting should only delay the first mail from a given sender to a given recipient, and the typical delay is in the order of minutes. Reality, however, can differ slightly. Some large ISPs use clusters of SMTP servers, and when a message is initially rejected, the server that retries the transmission may not be the same as the initial one. When that happens, the second server gets a temporary error message due to greylisting too, and so on; it may take several hours until transmission is attempted by a server that has already been involved, since SMTP servers usually increase the delay between retries at each failure.

As a consequence, the incoming IP address may vary in time even for a single sender. But it goes further: even the sender address can change. For instance, many mailing-list servers encode extra information in the sender address so as to be able to handle error messages (known as *bounces*). Each new message sent to a mailing-list may then need to go through greylisting, which means it has to be stored (temporarily) on the sender's server. For very large mailing-lists (with tens of thousands of subscribers), this can soon become a problem.

To mitigate these drawbacks, Postgrey manages a whitelist of such sites, and messages emanating from them are immediately accepted without going through greylisting. This list can easily be adapted to local needs, since it is stored in the `/etc/postgrey/whitelist_clients` file.

GOING FURTHER

Selective greylisting with *milter-greylist*

The drawbacks of greylisting can be mitigated by only using greylisting on the subset of clients that are already considered as probable sources of spam (because they are listed in a DNS blacklist). This is not possible with *postgrey* but *milter-greylist* can be used in such a way.

In that scenario, since DNS blacklists never triggers a definitive rejection, it becomes reasonable to use aggressive blacklists, including those listing all dynamic IP addresses from ISP clients (such as `pbl.spamhaus.org` or `dul.dnsbl.sorbs.net`).

Since *milter-greylist* uses Sendmail's milter interface, the postfix side of its configuration is limited to "`smtpd_milters = unix:/var/run/milter-greylist/milter-greylist.sock`". The `greylist.conf(5)` manual page documents `/etc/milter-greylist/greylist.conf` and the numerous ways to configure milter-greylist. You will also have to edit `/etc/default/milter-greylist` to actually enable the service.

11.1.5. Customizing Filters Based On the Recipient

section 11.1.3, "Restrictions for Receiving and Sending" page 277 and section 11.1.4, "Setting Up *greylisting*" page 283 reviewed many of the possible restrictions. They all have their use in limiting the amount of received spam, but they also all have their drawbacks. It is therefore more and more common to customize the set of filters depending on the recipient. At Falcot Corp, greylisting is interesting for most users, but it hinders the work of some users who need low latency in their emails (such as the technical support service). Similarly, the commercial service sometimes has problems receiving emails from some Asian providers who may be listed in blacklists; this service asked for a non-filtered address so as to be able to correspond.

Postfix provides such a customization of filters with a "restriction class" concept. The classes are declared in the smtpd_restriction_classes parameter, and defined the same way as smtpd_recipient_restrictions. The check_recipient_access directive then defines a table mapping a given recipient to the appropriate set of restrictions.

Example 11.9 *Defining restriction classes in `main.cf`*

```
smtpd_restriction_classes = greylisting, aggressive, permissive

greylisting = check_policy_service inet:127.0.0.1:10023
aggressive =
        reject_rbl_client sbl-xbl.spamhaus.org,
        check_policy_service inet:127.0.0.1:10023
permissive = permit

smtpd_recipient_restrictions =
        permit_mynetworks,
        reject_unauth_destination,
        check_recipient_access hash:/etc/postfix/recipient_access
```

Example 11.10 *The `/etc/postfix/recipient_access` file*

```
# Unfiltered addresses
postmaster@falcot.com  permissive
support@falcot.com     permissive
sales-asia@falcot.com  permissive

# Aggressive filtering for some privileged users
joe@falcot.com         aggressive

# Special rule for the mailing-list manager
sympa@falcot.com       reject_unverified_sender

# Greylisting by default
```

```
falcot.com              greylisting
```

11.1.6. Integrating an Antivirus

The many viruses circulating as attachments to emails make it important to set up an antivirus at the entry point of the company network, since despite an awareness campaign, some users will still open attachments from obviously shady messages.

SECURITY

Controversial Discussion of Anti-Virus Software

The usage of virus scanners, or so called antivirus software, is controversial. There is usually a gap between the release of some piece of malware and the addition of detection rules to the antivirus database. During this gap, there is no software-based protection. Further, the usage often requires to run additional software, for example, to uncompress archives and scan all kinds of executables, which drastically increases the exploit potential of the antivirus software itself. Usage of such software solutions can therefor never replace awareness campaigns and simple behavioral rules (never open unsolicited sent attachments, etc.).

The Falcot administrators selected `clamav` for their free antivirus. The main package is *clamav*, but they also installed a few extra packages such as *arj*, *unzoo*, *unrar* and *lha*, since they are required for the antivirus to analyze attachments archived in one of these formats.

The task of interfacing between antivirus and the email server goes to `clamav-milter`. A *milter* (short for *mail filter*) is a filtering program specially designed to interface with email servers. A milter uses a standard application programming interface (API) that provides much better performance than filters external to the email servers. Milters were initially introduced by *Sendmail*, but *Postfix* soon followed suit.

QUICK LOOK

A milter for Spamassassin

The *spamass-milter* package provides a milter based on *SpamAssassin*, the famous unsolicited email detector. It can be used to flag messages as probable spams (by adding an extra header) and/or to reject the messages altogether if their "spamminess" score goes beyond a given threshold.

Once the *clamav-milter* package is installed, the milter should be reconfigured to run on a TCP port rather than on the default named socket. This can be achieved with `dpkg-reconfigure clamav-milter`. When prompted for the "Communication interface with Sendmail", answer "inet:10002@127.0.0.1".

NOTE

Real TCP port vs named socket

The reason why we use a real TCP port rather than the named socket is that the postfix daemons often run chrooted and do not have access to the directory hosting the named socket. You could also decide to keep using a named socket and pick a location within the chroot (`/var/spool/postfix/`).

The standard ClamAV configuration fits most situations, but some important parameters can still be customized with `dpkg-reconfigure clamav-base`.

The last step involves telling Postfix to use the recently-configured filter. This is a simple matter of adding the following directive to `/etc/postfix/main.cf`:

```
# Virus check with clamav-milter
smtpd_milters = inet:[127.0.0.1]:10002
```

If the antivirus causes problems, this line can be commented out, and `systemctl reload postfix` should be run so that this change is taken into account.

IN PRACTICE

Testing the antivirus

Once the antivirus is set up, its correct behavior should be tested. The simplest way to do that is to send a test email with an attachment containing the `eicar.com` (or `eicar.com.zip`) file, which can be downloaded online:

➨ https://2016.eicar.org/86-0-Intended-use.html

This file is not a true virus, but a test file that all antivirus software on the market diagnose as a virus to allow checking installations.

All messages handled by Postfix now go through the antivirus filter.

11.1.7. Fighting Spam with SPF, DKIM and DMARC

The high number of unsolicited email sent every day led to the creation of several standards, which aim at validating, that the sending host of an email is authorized and that the email has not been tampered with. The following systems are all DNS-based and require the administrators to not only have control over the mail server, but over the DNS for the domain in question too.

CAUTION

Controversial Discussion

Like any other tool, the following standards have limits and real effects if put to use. They can (and should) lead to emails being rejected or even just discarded. If that happens to some legitimate emails (sometimes sent from a misconfigured SMTP server), it usually causes anger and a lack of understanding by the user. Therefor these rules are often applied as a ”soft fail” or a ”soft reject”, which usually means, that failing the checks only leads to adding a (header) mark to the affected email. There are people who think that this makes these standards ”broken by design”. Decide for yourself and be careful about how strict you choose to apply these standards.

Integrating the Sender Policy Framework (SPF)

The Sender Policy Framework (SPF) is used to validate if a certain mail server is allowed to send emails for a given domain. It is mostly configured through DNS. The syntax for the entry to make is explained in detail at:

➨ http://www.open-spf.org/SPF_Record_Syntax

➨ https://tools.ietf.org/html/rfc7208

➨ https://en.wikipedia.org/wiki/Sender_Policy_Framework

The following is a sample DNS entry which states that all the domain's Mail Exchange Resource Records (MX-RRs) are allowed to email the current domain, and all others are prohibited. The DNS entry does not need to be given a name. But to use the include directive it must have one.

```
Name: example.org
Type: TXT
TTL:  3600
Data: v=spf1 a mx -all
```

Let's take a quick look at the falcot.org entry.

```
# host -t TXT falcot.org
falcot.org descriptive text "v=spf1 ip4:199.127.61.96 +a +mx +ip4:206.221.184.234 +ip4:209.222.96.251 ~all"
```

It states, that the IP of the sender must match the A record for the sending domain, or must be listed as one of the Mail Exchange Resource Records for the current domain, or must be one of the three mentioned IP4 addresses. All other hosts should be marked as not being allowed to send email for the sender domain. The latter is called a "soft fail" and is intended to mark the email accordingly, but still accept it.

The `postfix` mail server can check the SPF record for incoming emails using the *postfix-policyd-spf-python* package, a policy agent written in Python. The file `/usr/share/doc/postfix-policyd-spf-python/README.Debian` describes the necessary steps to integrate the agent into postfix, so we won't repeat it here.

The configuration is done in the file `/etc/postfix-policyd-spf-python/policyd-spf.conf`, which is fully documented in `policyd-spf.conf(5)` and `/usr/share/doc/postfix-policyd-spf-python/policyd-spf.conf.commented.gz`. The main configuration parameters are HELO_reject and Mail_From_reject, which configure if emails should be rejected (Fail) or accepted with a header being appended (False), if checks fail. The latter is often useful, when the message is further processed by a spam filter.

If the result is intended to be used by *opendmarc* (section 11.1.7.3, "Integrating Domain-based Message Authentication, Reporting and Conformance (DMARC)" page 290), then Header_Type must be set to AR.

Note, that *spamassassin* contains a plugin to check the SPF record.

Integrating DomainKeys (DKIM) Signing and Checking

The Domain Keys Identified Mail (DKIM) standard is a sender authentication system. The mail transport agent, here `postfix`, adds a digital signature associated with the domain name to the header of outgoing emails. The receiving party can validate the message body and header fields by checking the signature against a public key, which is retrieved from the senders DNS records.

➨ http://dkim.org/

The necessary tools are shipped with the *opendkim* and *opendkim-tools* packages.

CAUTION

Mailing List Software and DKIM

Mailing list managers often rewrite some email headers, thus leading to invalid DKIM signatures. Even using a `relaxed` canonicalization does not always prevent this from happening. So the administrators must pay close attention to the mail severs log files to identify such issues. Otherwise such emails might be flagged as spam and might get rejected.

First the private key must be created using the command `opendkim-genkey -s` *`SELECTOR`* `-d` *`DOMAIN`*. *SELECTOR* must be a unique name for the key. It can be as simple as "mail" or the date of creation, if you plan to rotate keys.

Example 11.11 *Create a private key for signing E-Mails from falcot.com*

```
# opendkim-genkey -s mail -d falcot.com -D /etc/dkimkeys
# chown opendkim.opendkim /etc/dkimkeys/mail.*
```

This will create the files `/etc/dkimkeys/mail.private` and `/etc/dkimkeys/mail.txt` and set the appropriate ownership. The first file contains the private key and the latter the public key, that needs to be added to the DNS:

```
Name: mail._domainkey
Type: TXT
TTL:  3600
Data: "v=DKIM1; h=sha256; k=rsa; s=email; p=[...]"
```

The *opendkim* package in Debian defaults to a keysize of 2048 bit. Unfortunately some DNS servers can only handle text entries with a maximum length of 255 characters, which is exceeded by the chosen default keysize. In this case use the option `-b 1024` to chose a smaller keysize. If `opendkim-testkey` succeeds, the entry has been successfully set up. The syntax of the entry is explained here:

➨ https://tools.ietf.org/html/rfc6376

➨ https://en.wikipedia.org/wiki/DKIM

To configure *opendkim*, SOCKET and RUNDIR must be chosen in `/etc/default/opendkim`. Please note that SOCKET must be accessible from postfix in its chrooted environment. The further configuration is done in `/etc/opendkim.conf`. The following is a configuration excerpt, which makes sure that the Domain "falcot.com" and all subdomains (SubDomain) are signed by the Selector "mail" and the single private key (KeyFile) `/etc/dkimkeys/mail.private`. The "relaxed" Canonicalization for both the header and the body tolerates mild modification (by a mailing list software, for example). The filter runs both in signing ("s") and verification ("v") Mode. If a signature fails to validate (On-BadSignature), the mail should be quarantined ("q").

```
[...]
Domain                  falcot.com
KeyFile                 /etc/dkimkeys/mail.private
```

```
Selector                mail

[...]
Canonicalization        relaxed/relaxed
Mode                    sv
On-BadSignature         q
SubDomains              yes

[...]
Socket                  inet:12345@localhost

[...]
UserID                  opendkim
```

It is also possible to use multiple selectors/keys (KeyTable), domains (SigningTable) and to specify internal or trusted hosts (InternalHosts, ExternalIgnoreList), which may send mail through the server as one of the signing domains without credentials.

The following directives in `/etc/postfix/main.cf` make postfix use the filter:

```
milter_default_action = accept
non_smtpd_milters = inet:localhost:12345
smtpd_milters = inet:localhost:12345
```

To differentiate signing and verification it is sometimes more useful to add the directives to the services in `/etc/postfix/master.cf` instead.

More information is available in the `/usr/share/doc/opendkim/` directory and the manual pages `opendkim(8)` and `opendkim.conf(5)`.

Note that *spamassassin* contains a plugin to check the DKIM record.

Integrating Domain-based Message Authentication, Reporting and Conformance (DMARC)

The Domain-based Message Authentication, Reporting and Conformance (DMARC) standard can be used to define a DNS TXT entry with the name _dmarc and the action, that should be taken, when emails, which contain your domain as sending host, fail to validate using DKIM and SPF.

➡ https://dmarc.org/overview/

Let's have a look at the entries of two large providers:

```
# host -t TXT _dmarc.gmail.com
_dmarc.gmail.com descriptive text "v=DMARC1; p=none; sp=quarantine; rua=mailto:mailauth-reports@google.com"
# host -t TXT _dmarc.yahoo.com
_dmarc.yahoo.com descriptive text "v=DMARC1; p=reject; pct=100; rua=mailto:dmarc_y_rua@yahoo.com;"
```

Yahoo has a strict policy to reject all emails pretending to be sent from a Yahoo account but missing or failing DKIM and SPF checks. Google Mail (Gmail) propagates a very relaxed policy, in which such messages from the main domain should still be accepted (p=none). For subdomains

they should be marked as spam (sp=quarantine). The addresses given in the rua key can be used to send aggregated DMARC reports to. The full syntax is explained here:

➡ https://tools.ietf.org/html/rfc7489

➡ https://en.wikipedia.org/wiki/DMARC

The postfix mail server can use this information too. The *opendmarc* package contains the necessary milter. Similar to *opendkim* SOCKET and RUNDIR must be chosen in /etc/default/opendmarc (for Unix sockets you must make sure, that they are inside the postfix chroot to be found). The configuration file /etc/opendmarc.conf contains detailed comments and is also explained in opendmarc.conf(5). By default, emails failing the DMARC validation are not rejected but flagged, by adding an appropriate header field. To change this, use RejectFailures true.

The milter is then added to smtpd_milters and non_smtpd_milters. If we configured the opendkim and opendmarc milters to run on ports 12345 and 54321, the entry in /etc/postfix/main.cf looks like this:

```
non_smtpd_milters = inet:localhost:12345,inet:localhost:54321
smtpd_milters = inet:localhost:12345,inet:localhost:54321
```

The milter can also be selectively applied to a service in /etc/postfix/master.cf instead.

11.1.8. Authenticated SMTP

Being able to send emails requires an SMTP server to be reachable; it also requires said SMTP server to send emails through it. For roaming users, this may need regularly changing the configuration of the SMTP client, since Falcot's SMTP server rejects messages coming from IP addresses apparently not belonging to the company. Two solutions exist: either the roaming user installs an SMTP server on their computer, or they still use the company server with some means of authenticating as an employee. The former solution is not recommended since the computer won't be permanently connected, and it won't be able to retry sending messages in case of problems; we will focus on the latter solution.

SMTP authentication in Postfix relies on SASL (*Simple Authentication and Security Layer*). It requires installing the *libsasl2-modules* and *sasl2-bin* packages, then registering a password in the SASL database for each user that needs authenticating on the SMTP server. This is done with the saslpasswd2 command, which takes several parameters. The -u option defines the authentication domain, which must match the smtpd_sasl_local_domain parameter in the Postfix configuration. The -c option allows creating a user, and -f allows specifying the file to use if the SASL database needs to be stored at a different location than the default (/etc/sasldb2).

```
# saslpasswd2 -u 'postconf -h myhostname' -f /var/spool/postfix/etc/sasldb2 -c jean
[... type jean's password twice ...]
```

Note that the SASL database was created in Postfix's directory. In order to ensure consistency, we also turn `/etc/sasldb2` into a symbolic link pointing at the database used by Postfix, with the `ln -sf /var/spool/postfix/etc/sasldb2 /etc/sasldb2` command.

Now we need to configure Postfix to use SASL. First the postfix user needs to be added to the sasl group, so that it can access the SASL account database. A few new parameters are also needed to enable SASL, and the smtpd_recipient_restrictions parameter needs to be configured to allow SASL-authenticated clients to send emails freely.

Example 11.12 *Enabling SASL in `/etc/postfix/main.cf`*

```
# Enable SASL authentication
smtpd_sasl_auth_enable = yes
# Define the SASL authentication domain to use
smtpd_sasl_local_domain = $myhostname
[...]
# Adding permit_sasl_authenticated before reject_unauth_destination
# allows relaying mail sent by SASL-authenticated users
smtpd_recipient_restrictions =
    permit_sasl_authenticated,
    permit_mynetworks,
    reject_unauth_destination,
[...]
```

It is usually a good idea to not send passwords over an unencrypted connection. *Postfix* allows to use different configurations for each port (service) it runs on. All these can be configured with different rules and directives in the `/etc/postfix/master.cf` file. To turn off authentication at all for port 25 (smtpd service) add the following directive:

```
smtp      inet  n       -       y       -       -       smtpd
    [..]
    -o smtpd_sasl_auth_enable=no
    [..]
```

If for some reason clients use an outdated AUTH command (some very old mail clients do), interoperability with them can be enabled using the broken_sasl_auth_clients directive.

EXTRA

Authenticated SMTP client

Most email clients are able to authenticate to an SMTP server before sending outgoing messages, and using that feature is a simple matter of configuring the appropriate parameters. If the client in use does not provide that feature, the workaround is to use a local Postfix server and configure it to relay email via the remote SMTP server. In this case, the local Postfix itself will be the client that authenticates with SASL. Here are the required parameters:

```
smtp_sasl_auth_enable = yes
smtp_sasl_password_maps = hash:/etc/postfix/sasl_passwd
relay_host = [mail.falcot.com]
```

```
mail falcot com   joe LyinIsji
```

11.2. Web Server (HTTP)

The Falcot Corp administrators decided to use the Apache HTTP server, included in Debian *Buster* at version 2.4.38.

ALTERNATIVE

Other web servers

Apache is merely the most widely-known (and widely-used) web server, but there are others; they can offer better performance under certain workloads, but this has its counterpart in the smaller number of available features and modules. However, when the prospective web server is built to serve static files or to act as a proxy, the alternatives, such as *nginx* and *lighttpd*, are worth investigating.

11.2.1. Installing Apache

Installing the *apache2* package is all that is needed. It contains all the modules, including the *Multi-Processing Modules* (MPMs) that affect how Apache handles parallel processing of many requests, which used to be provided in separate *apache2-mpm-** packages. It will also pull *apache2-utils* containing the command line utilities that we will discover later.

The MPM in use affects significantly the way Apache will handle concurrent requests. With the *worker* MPM, it uses *threads* (lightweight processes), whereas with the *prefork* MPM it uses a pool of processes created in advance. With the *event* MPM it also uses threads, but the inactive connections (notably those kept open by the HTTP *keep-alive* feature) are handed back to a dedicated management thread.

The Falcot administrators also install *libapache2-mod-php7.3* so as to include the PHP support in Apache. This causes the default *event* MPM to be disabled, and *prefork* to be used instead. To use the *event* MPM one can use *php7.3-fpm*.

SECURITY

Execution under the `www-data` user

By default, Apache handles incoming requests under the identity of the `www-data` user. This means that a security vulnerability in a CGI script executed by Apache (for a dynamic page) won't compromise the whole system, but only the files owned by this particular user.

Using the *suexec* modules, provided by *apache2-suexec-** packages, allows bypassing this rule so that some CGI scripts are executed under the identity of another user. This is configured with a `SuexecUserGroup` *`usergroup`* directive in the Apache configuration.

Another possibility is to use a dedicated MPM, such as the one provided by *libapache2-mpm-itk*. This particular one has a slightly different behavior: it allows "isolating" virtual hosts (actually, sets of pages) so that they each run as a different user. A vulnerability in one website therefore cannot compromise files belonging to the owner of another website.

QUICK LOOK

List of modules

The full list of Apache standard modules can be found online.

➨ https://httpd.apache.org/docs/2.4/mod/index.html

Apache is a modular server, and many features are implemented by external modules that the main program loads during its initialization. The default configuration only enables the most common modules, but enabling new modules is a simple matter of running `a2enmod` *`module`*; to disable a module, the command is `a2dismod` *`module`*. These programs actually only create (or delete) symbolic links in `/etc/apache2/mods-enabled/`, pointing at the actual files (stored in `/etc/apache2/mods-available/`).

IN PRACTICE

Checking the configuration

The `mod_info` module (`a2enmod info`) allows to access the comprehensive Apache server configuration and information via browser visiting `http://localhost/server-info`. Because it might contain sensitive information, access is only allowed from the local host by default.

➨ https://httpd.apache.org/docs/2.4/mod/mod_info.html

With its default configuration, the web server listens on port 80 (as configured in `/etc/apache2/ports.conf`), and serves pages from the `/var/www/html/` directory (as configured in `/etc/apache2/sites-enabled/000-default.conf`).

11.2.2. Adding support for SSL

Apache 2.4 includes the SSL module (mod_ssl) required for secure HTTP (HTTPS) out of the box. It just needs to be enabled with `a2enmod ssl`, then the required directives have to be added to the configuration files. A configuration example is provided in `/etc/apache2/sites-available/default-ssl.conf`.

➨ https://httpd.apache.org/docs/2.4/mod/mod_ssl.html

If you want to generate trusted certificates, you can follow section section 10.2.1, "Creating gratis trusted certificates" page 240 and then adjust the following variables:

```
SSLCertificateFile      /etc/letsencrypt/live/DOMAIN/fullchain.pem
SSLCertificateKeyFile   /etc/letsencrypt/live/DOMAIN/privkey.pem
SSLCertificateChainFile /etc/letsencrypt/live/DOMAIN/chain.pem
SSLCACertificateFile    /etc/ssl/certs/ca-certificates.crt
```

Some extra care must be taken if you want to favor SSL connections with *Perfect Forward Secrecy* (those connections use ephemeral session keys ensuring that a compromission of the server's

secret key does not result in the compromission of old encrypted traffic that could have been stored while sniffing on the network). Have a look at Mozilla's recommendations in particular:

➨ `https://wiki.mozilla.org/Security/Server_Side_TLS#Apache`

As an alternative to the standard SSL module, there is an extension module called mod_gnutls, which is shipped with the *libapache2-mod-gnutls* package and enabled with the `a2enmod gnutls`.

➨ `https://mod.gnutls.org/`

11.2.3. Configuring Virtual Hosts

A virtual host is an extra identity for the web server.

Apache considers two different kinds of virtual hosts: those that are based on the IP address (or the port), and those that rely on the domain name of the web server. The first method requires allocating a different IP address (or port) for each site, whereas the second one can work on a single IP address (and port), and the sites are differentiated by the hostname sent by the HTTP client (which only works in version 1.1 of the HTTP protocol — fortunately that version is old enough that all clients use it already).

The (increasing) scarcity of IPv4 addresses usually favors the second method; however, it is made more complex if the virtual hosts need to provide HTTPS too, since the SSL protocol hasn't always provided for name-based virtual hosting; the SNI extension (*Server Name Indication*) that allows such a combination is not handled by all browsers. When several HTTPS sites need to run on the same server, they will usually be differentiated either by running on a different port or on a different IP address (IPv6 can help there).

The default configuration for Apache 2 enables name-based virtual hosts. In addition, a default virtual host is defined in the /etc/apache2/sites-enabled/000-default.conf file; this virtual host will be used if no host matching the request sent by the client is found.

CAUTION

First virtual host

Requests concerning unknown virtual hosts will always be served by the first defined virtual host, which is why we defined `www.falcot.com` first here.

QUICK LOOK

Apache supports SNI

The Apache server supports an SSL protocol extension called *Server Name Indication* (SNI). This extension allows the browser to send the hostname of the web server during the establishment of the SSL connection, much earlier than the HTTP request itself, which was previously used to identify the requested virtual host among those hosted on the same server (with the same IP address and port). This allows Apache to select the most appropriate SSL certificate for the transaction to proceed.

Before SNI, Apache would always use the certificate defined in the default virtual host. Clients trying to access another virtual host would then display warnings, since the certificate they received didn't match the website they were trying to access. Fortunately, most browsers now work with SNI; this includes Microsoft Internet Explorer starting with version 7.0 (starting on Vista), Mozilla Firefox starting with version 2.0, Apple Safari since version 3.2.1, and all versions of Google Chrome.

The Apache package provided in Debian is built with support for SNI; no particular configuration is therefore needed.

Each extra virtual host is then described by a file stored in `/etc/apache2/sites-available/`. Setting up a website for the falcot.org domain is therefore a simple matter of creating the following file, then enabling the virtual host with `a2ensite www.falcot.org`.

Example 11.13 *The `/etc/apache2/sites-available/www.falcot.org.conf` file*

```
<VirtualHost *:80>
ServerName   www.falcot.org
ServerAlias  falcot.org
DocumentRoot /srv/www/www.falcot.org
</VirtualHost>
```

The Apache server, as configured so far, uses the same log files for all virtual hosts (although this could be changed by adding CustomLog directives in the definitions of the virtual hosts). It therefore makes good sense to customize the format of this log file to have it include the name of the virtual host. This can be done by creating a `/etc/apache2/conf-available/customlog.conf` file that defines a new format for all log files (with the LogFormat directive) and by enabling it with `a2enconf customlog`. The CustomLog line must also be removed (or commented out) from the `/etc/apache2/sites-available/000-default.conf` file.

Example 11.14 *The `/etc/apache2/conf-available/customlog.conf` file*

```
# New log format including (virtual) host name
LogFormat "%v %h %l %u %t \"%r\" %>s %b \"%{Referer}i\" \"%{User-Agent}i\"" vhost

# Now let's use this "vhost" format by default
CustomLog /var/log/apache2/access.log vhost
```

11.2.4. Common Directives

This section briefly reviews some of the commonly-used Apache configuration directives.

The main configuration file usually includes several Directory blocks; they allow specifying different behaviors for the server depending on the location of the file being served. Such a block commonly includes Options and AllowOverride directives.

Example 11.15 *Directory block*

```
<Directory /srv/www>
Options Includes FollowSymlinks
```

```
AllowOverride All
DirectoryIndex index.php index.html index.htm
</Directory>
```

The DirectoryIndex directive contains a list of files to try when the client request matches a directory. The first existing file in the list is used and sent as a response.

The Options directive is followed by a list of options to enable. The None value disables all options; correspondingly, All enables them all except MultiViews. Available options include:

- ExecCGI indicates that CGI scripts can be executed.
- FollowSymlinks tells the server that symbolic links can be followed, and that the response should contain the contents of the target of such links.
- SymlinksIfOwnerMatch also tells the server to follow symbolic links, but only when the link and the its target have the same owner.
- Includes enables *Server Side Includes* (*SSI* for short). These are directives embedded in HTML pages and executed on the fly for each request.
- IncludesNOEXEC allows *Server Side Includes* (*SSI*) but disables the exec command and limits the include directive to text/markup files.
- Indexes tells the server to list the contents of a directory if the HTTP request sent by the client points at a directory without an index file (i.e., when no files mentioned by the DirectoryIndex directive exists in this directory).
- MultiViews enables content negotiation; this can be used by the server to return a web page matching the preferred language as configured in the browser.

BACK TO BASICS

.htaccess file

The `.htaccess` file contains Apache configuration directives enforced each time a request concerns an element of the directory where it is stored. The scope of these directives also recurses to all the subdirectories within.

Most of the directives that can occur in a `Directory` block are also legal in a `.htaccess` file.

The AllowOverride directive lists all the options that can be enabled or disabled by way of a `.htaccess` file. A common use of this option is to restrict ExecCGI, so that the administrator chooses which users are allowed to run programs under the web server's identity (the www-data user).

Requiring Authentication

In some circumstances, access to part of a website needs to be restricted, so only legitimate users who provide a username and a password are granted access to the contents.

Example 11.16 *.htaccess file requiring authentication*

```
Require valid-user
AuthName "Private directory"
AuthType Basic
AuthUserFile /etc/apache2/authfiles/htpasswd-private
```

SECURITY

No security

The authentication system used in the above example (`Basic`) has minimal security as the password is sent in clear text (it is only encoded as *base64*, which is a simple encoding rather than an encryption method). It should also be noted that the documents "protected" by this mechanism also go over the network in the clear. If security is important, the whole HTTP connection should be encrypted with SSL.

The `/etc/apache2/authfiles/htpasswd-private` file contains a list of users and passwords; it is commonly manipulated with the `htpasswd` command. For example, the following command is used to add a user or change their password:

```
# htpasswd /etc/apache2/authfiles/htpasswd-private user
New password:
Re-type new password:
Adding password for user user
```

Restricting Access

The Require directive controls access restrictions for a directory (and its subdirectories, recursively).

➨ https://httpd.apache.org/docs/2.4/howto/access.html

It can be used to restrict access based on many criteria; we will stop at describing access restriction based on the IP address of the client, but it can be made much more powerful than that, especially when several Require directives are combined within a RequireAll block.

Example 11.17 *Only allow from the local network*

```
Require ip 192.168.0.0/16
```

ALTERNATIVE

Old syntax

The `Require` syntax is only available in Apache 2.4 (the version shipped since *Jessie*). For users of *Wheezy*, the Apache 2.2 syntax is different, and we describe it here mainly for reference, although it can also be made available in Apache 2.4 using the `mod_access_compat` module.

The `Allow from` and `Deny from` directives control access restrictions for a directory (and its subdirectories, recursively).

The `Order` directive tells the server of the order in which the `Allow from` and `Deny from` directives are applied; the last one that matches takes precedence. In concrete terms, `Order deny,allow` allows access if no `Deny from` applies, or if an `Allow from` directive does. Conversely, `Order allow,deny` rejects access if no `Allow from` directive matches (or if a `Deny from` directive applies).

The `Allow from` and `Deny from` directives can be followed by an IP address, a network (such as `192.168.0.0/255.255.255.0`, `192.168.0.0/24` or even `192.168.0`), a hostname or a domain name, or the `all` keyword, designating everyone.

For instance, to reject connections by default but allow them from the local network, you could use this:

```
Order deny,allow
Allow from 192.168.0.0/16
Deny from all
```

11.2.5. Log Analyzers

A log analyzer is frequently installed on a web server; since the former provides the administrators with a precise idea of the usage patterns of the latter.

The Falcot Corp administrators selected *AWStats* (*Advanced Web Statistics*) to analyze their Apache log files.

The first configuration step is the customization of the `/etc/awstats/awstats.conf` file. The Falcot administrators keep it unchanged apart from the following parameters:

```
LogFile="/var/log/apache2/access.log"
LogFormat = "%virtualname %host %other %logname %time1 %methodurl %code %bytesd %
    ➥ refererquot %uaquot"
SiteDomain="www.falcot.com"
HostAliases="falcot.com REGEX[^.*\.falcot\.com$]"
DNSLookup=1
LoadPlugin="tooltips"
```

All these parameters are documented by comments in the template file. In particular, the `LogFile` and `LogFormat` parameters describe the location and format of the log file and the information it contains; `SiteDomain` and `HostAliases` list the various names under which the main web site is known.

For high traffic sites, `DNSLookup` should usually not be set to `1`; for smaller sites, such as the Falcot one described above, this setting allows getting more readable reports that include full machine names instead of raw IP addresses.

SECURITY

Access to statistics

AWStats makes its statistics available on the website with no restrictions by default, but restrictions can be set up so that only a few (probably internal) IP addresses can access them; the list of allowed IP addresses needs to be defined in the `AllowAccessFromWebToFollowingIPAddresses` parameter

AWStats will also be enabled for other virtual hosts; each virtual host needs its own configuration file, such as `/etc/awstats/awstats.www.falcot.org.conf`.

Example 11.18 *AWStats configuration file for a virtual host*

```
Include "/etc/awstats/awstats.conf"
SiteDomain="www.falcot.org"
HostAliases="falcot.org"
```

AWStats uses many icons stored in the `/usr/share/awstats/icon/` directory. In order for these icons to be available on the web site, the Apache configuration needs to be adapted to include the following directive:

```
Alias /awstats-icon/ /usr/share/awstats/icon/
```

After a few minutes (and once the script has been run a few times), the results are available online:

- http://www.falcot.com/cgi-bin/awstats.pl
- http://www.falcot.org/cgi-bin/awstats.pl

CAUTION

Log file rotation

In order for the statistics to take all the logs into account, *AWStats* needs to be run right before the Apache log files are rotated. Looking at the `prerotate` directive of `/etc/logrotate.d/apache2` file, this can be solved by putting a symlink to `/usr/share/awstats/tools/update.sh` in `/etc/logrotate.d/httpd-prerotate`:

```
$ cat /etc/logrotate.d/apache2
/var/log/apache2/*.log {
  daily
  missingok
  rotate 14
  compress
  delaycompress
  notifempty
  create 644 root adm
  sharedscripts
  postrotate
    if invoke-rc.d apache2 status > /dev/null 2>&1; then \
      invoke-rc.d apache2 reload > /dev/null 2>&1; \
    fi;
  endscript
```

```
  prerotate
    if [ -d /etc/logrotate.d/httpd-prerotate ]; then \
      run-parts /etc/logrotate.d/httpd-prerotate; \
    fi; \
  endscript
}

$ sudo mkdir -p /etc/logrotate.d/httpd-prerotate
$ sudo ln -sf /usr/share/awstats/tools/update.sh \
  /etc/logrotate.d/httpd-prerotate/awstats
```

Note also that the log files created by `logrotate` need to be readable by everyone, especially AWStats. In the above example, this is ensured by the `create 644 root adm` line (instead of the default `640` permissions).

11.3. FTP File Server

FTP (*File Transfer Protocol*) is one of the first protocols of the Internet (RFC 959 was issued in 1985!). It was used to distribute files before the Web was even born (the HTTP protocol was created in 1990, and formally defined in its 1.0 version by RFC 1945, issued in 1996).

This protocol allows both file uploads and file downloads; for this reason, it is still widely used to deploy updates to a website hosted by one's Internet service provider (or any other entity hosting websites). In these cases, secure access is enforced with a user identifier and password; on successful authentication, the FTP server grants read-write access to that user's home directory.

Other FTP servers are mainly used to distribute files for public downloading; Debian packages are a good example. The contents of these servers is fetched from other, geographically remote, servers; it is then made available to less distant users. This means that client authentication is not required; as a consequence, this operating mode is known as "anonymous FTP". To be perfectly correct, the clients do authenticate with the anonymous username; the password is often, by convention, the user's email address, but the server ignores it.

Many FTP servers are available in Debian (*ftpd*[1], *proftpd-basic*, *pyftpd* and so on). The Falcot Corp administrators picked *vsftpd* because they only use the FTP server to distribute a few files (including a Debian package repository); since they don't need advanced features, they chose to focus on the security aspects.

Installing the package creates an ftp system user. This account is always used for anonymous FTP connections, and its home directory (`/srv/ftp/`) is the root of the tree made available to users connecting to this service. The default configuration (in `/etc/vsftpd.conf`) requires some changes to cater to the simple need of making big files available for public downloads:

[1] The *ftpd* package is not included in Debian *Buster* due to a bug, which could not be solved before the release.

anonymous access needs to be enabled (anonymous_enable=YES) and read-only access of local users needs to be disabled (local_enable=NO). The latter is particularly important since the FTP protocol doesn't use any form of encryption and the user password could be intercepted over the wire.

11.4. NFS File Server

NFS (*Network File System*) is a protocol allowing remote access to a filesystem through the network. All Unix systems can work with this protocol.

SPECIFIC CASE

Microsoft Windows and NFS Shares

When older or (so called) "Home" variants of Windows are involved, usually Samba (section 11.5, "Setting Up Windows Shares with Samba" page 305) must be used instead of NFS. Modern Windows Server and "Pro" or "Enterprise" Desktop solutions however have built-in support for NFS. After installation of the "Services for NFS" components NFS shares can be accessed and temporarily or permanently mounted like any other network share. Be aware of possible encoding issues in file names.

As an alternative Debian can be installed on Windows 10 Pro and higher. It requires the installation of the Windows Subsystem for Linux component and the Debian app from the Windows store.

➨ https://www.microsoft.com/en-us/p/debian/9msvkqc78pk6?

NFS is a very useful tool but, historically, it has suffered from many limitations, most of which have been addressed with version 4 of the protocol. The downside is that the latest version of NFS is harder to configure when you want to make use of basic security features such as authentication and encryption since it relies on Kerberos for those parts. And without those, the NFS protocol must be restricted to a trusted local network since data goes over the network unencrypted (a *sniffer* can intercept it) and access rights are granted based on the client's IP address (which can be spoofed).

DOCUMENTATION

NFS HOWTO

Good documentation to deploy NFSv4 is rather scarce. Here are some pointers with content of varying quality but that should at least give some hints on what should be done.

➨ https://help.ubuntu.com/community/NFSv4Howto

➨ https://wiki.linux-nfs.org/wiki/index.php/Nfsv4_configuration

11.4.1. Securing NFS

If you don't use the Kerberos-based security features, it is vital to ensure that only the machines allowed to use NFS can connect to the various required RPC servers, because the basic protocol trusts the data received from the network. The firewall must also block *IP spoofing* so as to prevent an outside machine from acting as an inside one, and access to the appropriate ports must be restricted to the machines meant to access the NFS shares.

BACK TO BASICS

RPC

RPC (*Remote Procedure Call*) is a Unix standard for remote services. NFS is one such service.

RPC services register to a directory known as the *portmapper*. A client wishing to perform an NFS query first addresses the *portmapper* (on port 111, either TCP or UDP), and asks for the NFS server; the reply usually mentions port 2049 (the default for NFS). Not all RPC services necessarily use a fixed port.

Older versions of the protocol required other RPC services which used dynamically assigned ports. Fortunately, with NFS version 4, only port 2049 (for NFS) and 111 (for the portmapper) are needed and they are thus easy to firewall.

11.4.2. NFS Server

The NFS server is part of the Linux kernel; in kernels provided by Debian it is built as a kernel module. If the NFS server is to be run automatically on boot, the *nfs-kernel-server* package should be installed; it contains the relevant start-up scripts.

The NFS server configuration file, `/etc/exports`, lists the directories that are made available over the network (*exported*). For each NFS share, only the given list of machines is granted access. More fine-grained access control can be obtained with a few options. The syntax for this file is quite simple:

```
/directory/to/share machine1(option1,option2,...) machine2(...) ...
```

Note that with NFSv4, all exported directories must be part of a single hierarchy and that the root directory of that hierarchy must be exported and identified with the option fsid=0 or fsid=root.

Each machine can be identified either by its DNS name or its IP address. Whole sets of machines can also be specified using either a syntax such as *.falcot.com or an IP address range such as 192.168.0.0/255.255.255.0 or 192.168.0.0/24.

Directories are made available as read-only by default (or with the ro option). The rw option allows read-write access. NFS clients typically connect from a port restricted to root (in other words, below 1024); this restriction can be lifted by the insecure option (the secure option is implicit, but it can be made explicit if needed for clarity).

By default, the server only answers an NFS query when the current disk operation is complete (sync option); this can be disabled with the async option. Asynchronous writes increase performance a bit, but they decrease reliability since there is a data loss risk in case of the server crashing between the acknowledgment of the write and the actual write on disk. Since the default value changed recently (as compared to the historical value of NFS), an explicit setting is recommended.

In order to not give root access to the filesystem to any NFS client, all queries appearing to come from a root user are considered by the server as coming from the nobody user. This behavior

corresponds to the root_squash option, and is enabled by default. The no_root_squash option, which disables this behavior, is risky and should only be used in controlled environments. If all users should be mapped to the user nobody, use all_squash. The anonuid=*uid* and anongid=*gid* options allow specifying another fake user to be used instead of UID/GID 65534 (which corresponds to user nobody and group nogroup).

With NFSv4, you can add a sec option to indicate the security level that you want: sec=sys is the default with no special security features, sec=krb5 enables authentication only, sec=krb5i adds integrity protection, and sec=krb5p is the most complete level which includes privacy protection (with data encryption). For this to work you need a working Kerberos setup (that service is not covered by this book).

Other options are available; they are documented in the `exports(5)` manual page.

CAUTION

First installation

The `/etc/init.d/nfs-kernel-server` boot script only starts the server if `/etc/exports` lists one or more valid NFS shares. On initial configuration, once this file has been edited to contain valid entries, the NFS server must therefore be started with the following command:

```
# systemctl start nfs-kernel-server
```

11.4.3. NFS Client

As with other filesystems, integrating an NFS share into the system hierarchy requires mounting (and the *nfs-common* package). Since this filesystem has its peculiarities, a few adjustments were required in the syntaxes of the `mount` command and the `/etc/fstab` file.

Example 11.19 *Manually mounting with the `mount` command*

```
# mount -t nfs4 -o rw,nosuid arrakis.internal.falcot.com:/shared /srv/shared
```

Example 11.20 *NFS entry in the `/etc/fstab` file*

```
arrakis.internal.falcot.com:/shared /srv/shared nfs4 rw,nosuid 0 0
```

The entry described above mounts, at system startup, the NFS directory `/shared/` from the arrakis server into the local `/srv/shared/` directory. Read-write access is requested (hence the rw parameter). The nosuid option is a protection measure that wipes any setuid or setgid bit from programs stored on the share. If the NFS share is only meant to store documents, another recommended option is noexec, which prevents executing programs stored on the share. Note that on the server, the `shared` directory is below the NFSv4 root export (for example `/export/shared`), it is not a top-level directory.

The `nfs(5)` manual page describes all the options in some detail.

11.5. Setting Up Windows Shares with Samba

Samba is a suite of tools handling the SMB protocol (also known as "CIFS") on Linux. This protocol is used by Windows for network shares and shared printers.

Samba can also act as a Windows domain controller. This is an outstanding tool for ensuring seamless integration of Linux servers and the office desktop machines still running Windows.

11.5.1. Samba Server

The *samba* package contains the main two servers of Samba 4, `smbd` and `nmbd`.

DOCUMENTATION

Going further

The Samba server is extremely configurable and versatile, and can address a great many different use cases matching very different requirements and network architectures. This book only focuses on the use case where Samba is used as a standalone server, but it can also be a NT4 Domain Controller or a full Active Directory Domain Controller, or a simple member of an existing domain (which could be a managed by a Windows server).

The *samba* package contains all the necessary manual pages and in `/usr/share/doc/samba/examples/` a wealth of commented example files. If you are looking for a more comprehensive documentation, you may check the *Samba* website.

➨ `https://www.samba.org/samba/docs/`

TOOL

Authenticating with a Windows Server

Winbind gives system administrators the option of using a Windows server as an authentication server. Winbind also integrates cleanly with PAM and NSS. This allows setting up Linux machines where all users of a Windows domain automatically get an account.

More information can be found in the `/usr/share/doc/libpam-winbind/examples/pam_winbind/` directory of the *libpam-winbind* package.

Configuring with `debconf`

The package sets up a minimal configuration during the initial installation by plainly copying `/usr/share/samba/smb.conf`. So you should really run `dpkg-reconfigure samba-common` to adapt it:

On first installation the only piece of required information is the name of the workgroup where the Samba server will belong (the answer is FALCOTNET in our case).

In case of a package update (from the old stable Debian version) or if the SMB server has already been configured to use a WINS server (wins server) the package also proposes identifying the

WINS server from the information provided by the DHCP daemon. The Falcot Corp administrators rejected this option, since they intend to use the Samba server itself as the WINS server.

Configuring Manually

Changes to `smb.conf` The requirements at Falcot require other options to be modified in the `/etc/samba/smb.conf` configuration file. The following excerpts summarize the changes that were effected in the [global] section.

```
[...]

[global]

## Browsing/Identification ###

# Change this to the workgroup/NT-domain name your Samba server will part of
   workgroup = FALCOTNET

# Windows Internet Name Serving Support Section:
# WINS Support - Tells the NMBD component of Samba to enable its WINS Server
   wins support = yes ❶

[...]

####### Authentication #######

# Server role. Defines in which mode Samba will operate. Possible
# values are "standalone server", "member server", "classic primary
# domain controller", "classic backup domain controller", "active
# directory domain controller".
#
# Most people will want "standalone server" or "member server".
# Running as "active directory domain controller" will require first
# running "samba-tool domain provision" to wipe databases and create a
# new domain.
   server role = standalone server

   obey pam restrictions = yes

[...]

# "security = user" is always a good idea. This will require a Unix account
# in this server for every user accessing the server.
   security = user ❷

[...]
```

❶ Indicates that Samba should act as a Netbios name server (WINS) for the local network. This option has been removed from the default configuration in *Buster* and must be added manually if desired.

❷ This is the default value for this parameter; however, since it is central to the Samba configuration, filling it explicitly is recommended. Each user must authenticate before accessing any share.

Adding Users Each Samba user needs an account on the server; the Unix accounts must be created first, then the user needs to be registered in Samba's database. The Unix step is done quite normally (using `adduser` for instance).

Adding an existing user to the Samba database is a matter of running the `smbpasswd -a user` command; this command asks for the password interactively.

A user can be deleted with the `smbpasswd -x user` command. A Samba account can also be temporarily disabled (with `smbpasswd -d user`) and re-enabled later (with `smbpasswd -e user`).

11.5.2. Samba Client

The client features in Samba allow a Linux machine to access Windows shares and shared printers. The required programs are available in the *cifs-utils* and *smbclient* packages.

The `smbclient` Program

The `smbclient` program queries SMB servers. It accepts a -U *user* option, for connecting to the server under a specific identity. `smbclient //server/share` accesses the share in an interactive way similar to the command-line FTP client. `smbclient -L server` lists all available (and visible) shares on a server.

Mounting Windows Shares

The `mount` command allows mounting a Windows share into the Linux filesystem hierarchy (with the help of `mount.cifs` provided by *cifs-utils*).

Example 11.21 *Mounting a Windows share*

```
mount -t cifs //arrakis/shared /shared \
      -o credentials=/etc/smb-credentials
```

The `/etc/smb-credentials` file (which must not be readable by users) has the following format: